
Universidad del Rosario
Escuela de Ingeniería, Ciencia y Tecnología
DEPE
PROGRAMACIÓN ORIENTADA A OBJETOS (POO) EN PYTHON
SÁBADO
15 DE MARZO
10:00 AM A 12:00 PM
Esta sesión de dos horas te brindará una introducción sólida a la Programación Orientada a Objetos (POO) en Python, un paradigma de programación que se basa en la creación de objetos, lo que permite organizar el código de manera más eficiente y reutilizable.
Taller Knuth
MODALIDAD ACCESO REMOTO
NO SE REQUIERE EXPERIENCIA PREVIA EN PROGRAMACIÓN
Universidad del Rosario
Escuela de Ingeniería, Ciencia y Tecnología
MACC

Sobre el conferencista



Miguel Angel Orjuela Rocha

Ing. de Sistemas y Computación

Skills:

- Desarrollo Fullstack
- Analítica de datos
- Transferencia de conocimiento

Contacto:

miguela.orjuela@urosario.edu.co

https://www.linkedin.com/in/miguel-orjuela/

https://github.com/maorjuela73

Contenido

- Introducción
- Entorno de desarrollo
- Ejemplo a modelar
- Definición de una nueva clase
- Atributos
- Operaciones o métodos
- Uso de instancias como parámetros
- Conversión de un objeto a string
- Instancias como valores de retorno
- Detalles del modelado

Introducción

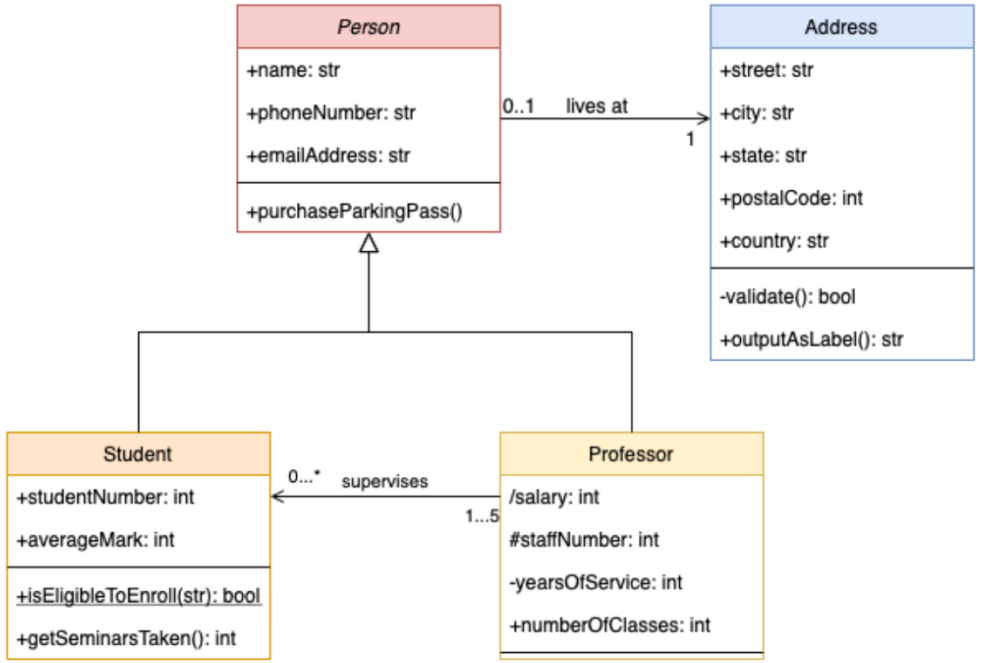
Objeto

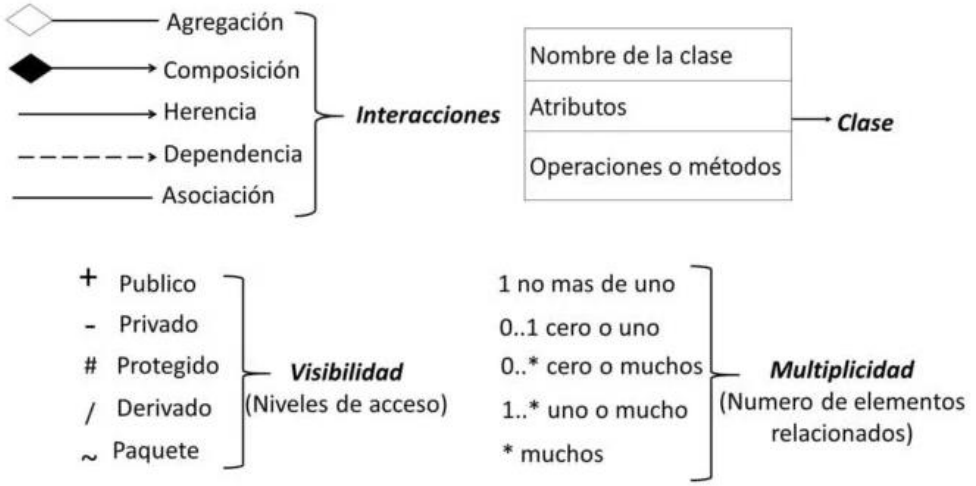
Bloque fundamental de la programación orientada a objetos.

- Es una colección de **datos** con **comportamientos** asociados.
- Contienen datos (**atributos**) y tienen funciones (**métodos**) para acceder y cambiar esos datos.

Clase

- Las clases describen objetos.
- Son como planos para crear un objeto.
- Se pueden representar gráficamente mediante el lenguaje gráfico de notación UML

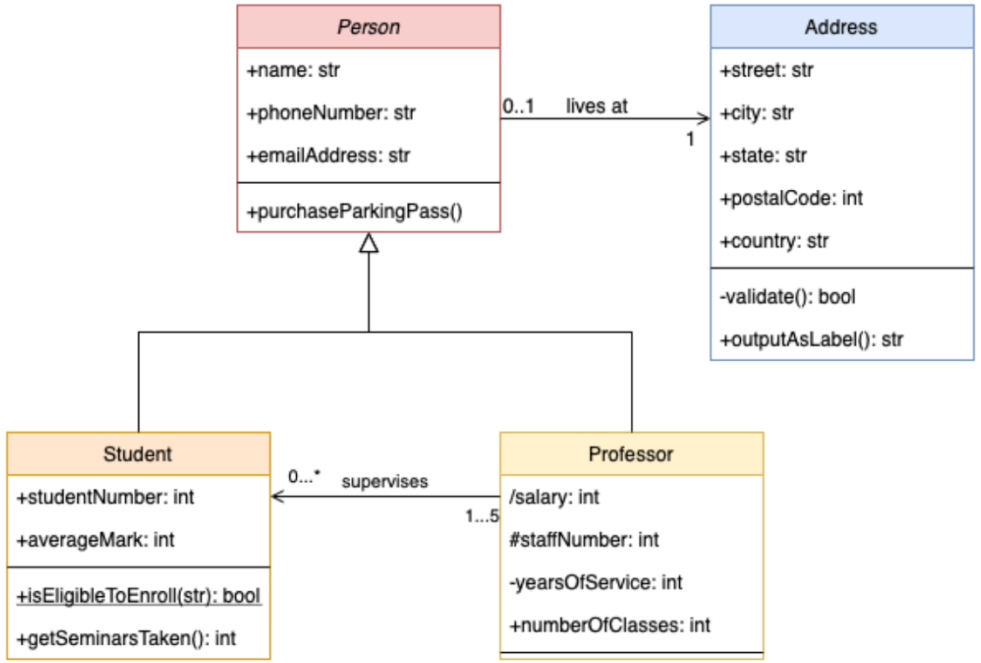


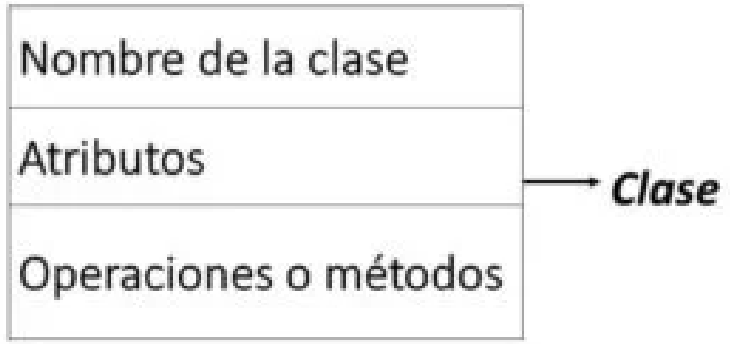


https://www.lucidchart.com/pages/uml-class-diagram

Clase

- Las clases describen objetos.
- Son como planos para crear un objeto.
- Se pueden representar gráficamente mediante el lenguaje gráfico de notación UML





https://www.lucidchart.com/pages/uml-class-diagram

Notación dentro del diagrama

Visibilidad (niveles de acceso)

Modificador de Acceso	Símbolo	Descripción
Público	+	Accesible desde cualquier clase.
Privado	–	Solo accesible dentro de la misma clase.
Protegido	#	Accesible dentro de la misma clase y sus subclases.
Paquete (Default)	~	Accesible solo dentro del mismo paquete (sin modificar).
Derivado	/	Indica miembros heredados de una clase base.
Estático	*subrayado*	Indica que el miembro pertenece a la clase y no a una instancia.

Multiplicidad

Multiplicidad	Símbolo	Descripción
0..1	`0..1`	Cero o una instancia.
1	`1`	Exactamente una instancia.
0..*	`0..*`	Cero o más instancias.
1..*	`1..*`	Al menos una instancia.
n	`n`	Exactamente n instancias.
m..n	`m..n`	Un rango de instancias, mínimo `m` y máximo n.

Interacciones

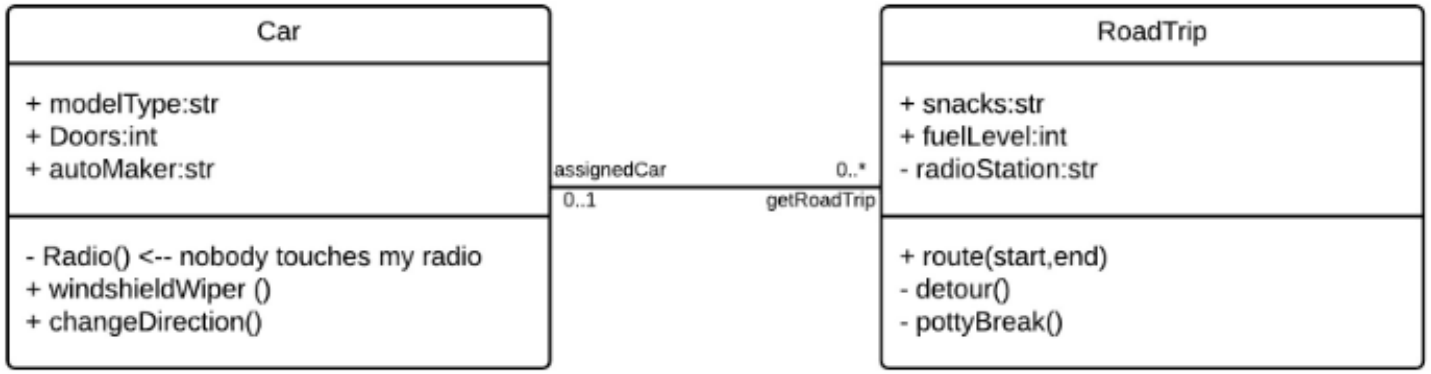
Relación	Símbolo UML	Descripción
Asociación	— (línea sólida)	Relación entre dos clases que interactúan entre sí. Puede ser unidireccional o bidireccional.
Dependencia	--> (línea discontinua)	Una clase usa temporalmente otra, pero no la posee.
Agregación	◇— (rombo vacío)	Relación "tiene un", donde una clase contiene otra, pero las instancias pueden existir por separado.
Composición	◆— (rombo lleno)	Relación "es parte de", donde una clase posee otra y controla su ciclo de vida.
Herencia	▲— (flecha hueca)	Una clase hija hereda atributos y métodos de una clase padre (generalización).

Relación	Posee la otra clase	¿Ciclo de vida compartido?	Dirección
Asociación	No necesariamente	No	Unidireccional o bidireccional
Dependencia	No	No	Unidireccional
Agregación	Sí (relación débil)	No	Unidireccional
Composición	Sí (relación fuerte)	Sí	Unidireccional
Herencia	Sí, hereda atributos y métodos	N/A	Unidireccional

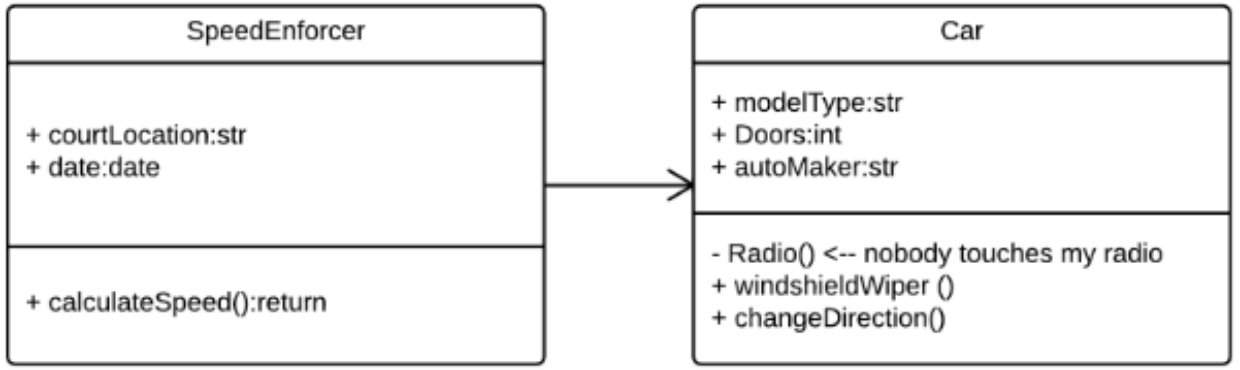
https://www.lucidchart.com/pages/uml-class-diagram

Asociaciones entre clases

- Bidireccional



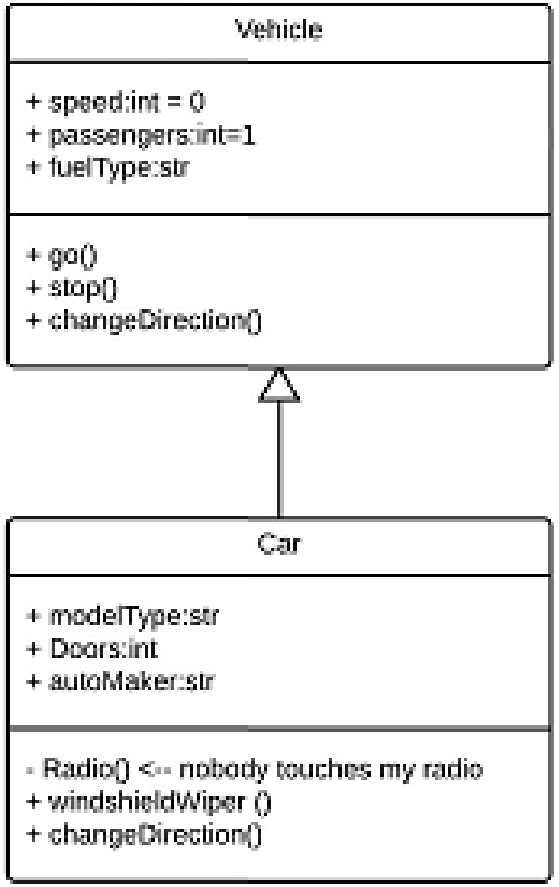
- Unidireccional



https://www.lucidchart.com/pages/uml-class-diagram

Herencia

- Proceso en el que una subclase (hija) hereda funcionalidades de una superclase (padre).
- También conocido como **generalización**.
- Permite reutilizar código y establecer una jerarquía de clases.



https://www.lucidchart.com/pages/uml-class-diagram

Entorno de desarrollo

Herramientas a usar

Entorno de desarrollo

Intérprete de Python

- https://www.python.org/

Entornos Integrados de Desarrollo (**IDEs**)

- https://www.spyder-ide.org/
- https://jupyter.org/try
- https://www.jetbrains.com/es-es/pycharm/
- https://code.visualstudio.com/
- https://www.sublimetext.com/
- …

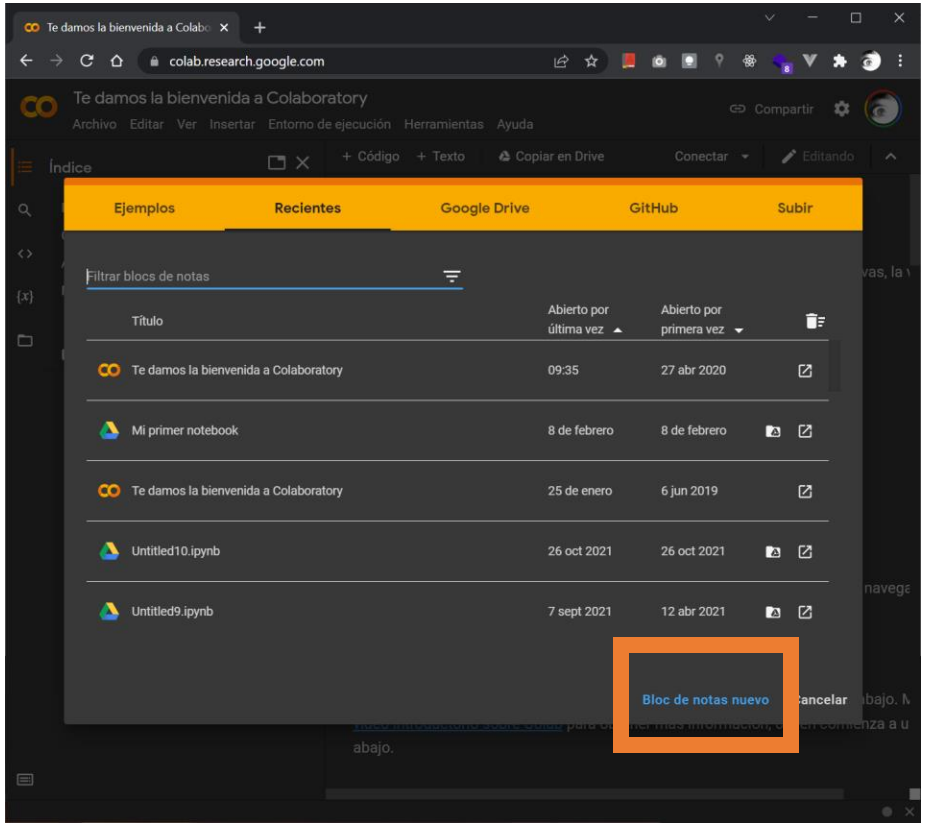


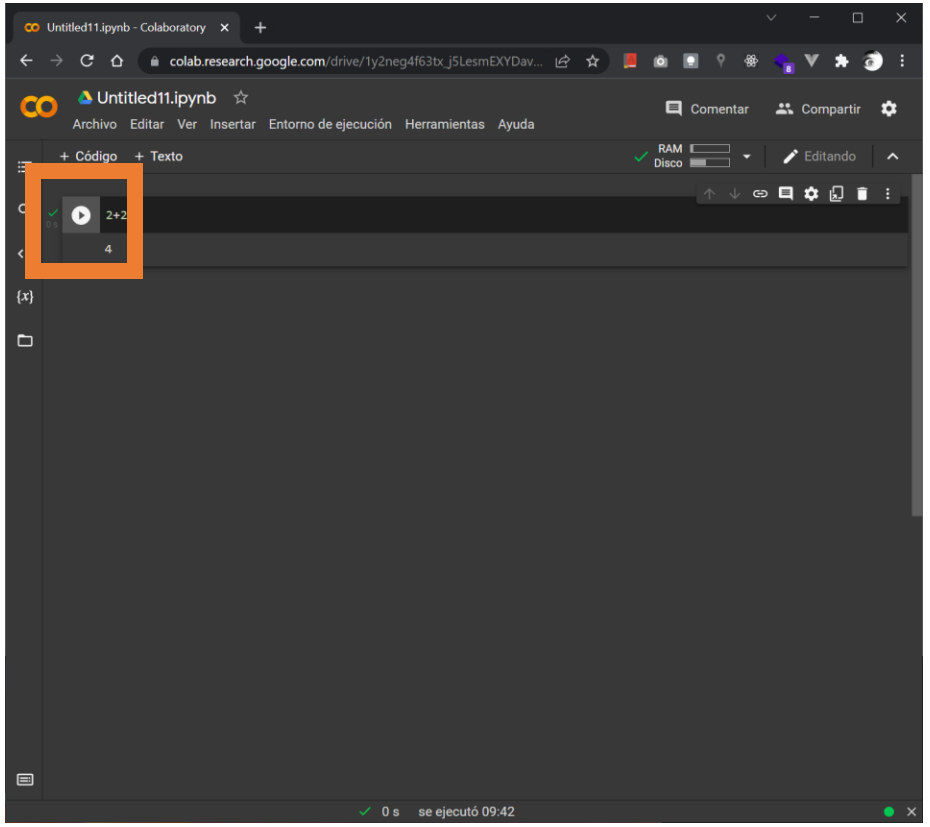
Entornos Cloud

- https://colab.research.google.com/

Ejecución de código en Colab

- Los blocs de notas funcionan con celdas
- Para ejecutar código de una celda se da “Play”





Tipos de datos: Números

Tipo	Ejemplo
`int`	`x = 42`
`float`	`y = 3.14`
`complex`	`z = 2 + 3j`

- Es posible convertir un dato a otro tipo

```
int(3.9)     # 3
float(5)     # 5.0
complex(2)   # (2+0j)
```

Tipos de datos: Texto

Tipo	Ejemplo
`str`	`s = "Hola"`

- Operaciones comunes

```
s = "Python"
s.lower()       # "python"
s.upper()       # "PYTHON"
s[0]            # "P"
s[-1]           # "n"
s[0:3]          # "Pyt"
len(s)          # 6
"Py" in s       # True
```

Tipos de datos: Lógicos (booleanos)

Tipo	Ejemplo
`bool`	`True`, `False`

```
bool(0)      # False
bool("")     # False
bool("x")    # True
```

Conversiones

Operador	Ejemplo	Resultado
`and`	`True and False`	`False`
`or`	`True or False`	`True`
`not`	`not True`	`False`

Operadores lógicos

Tipos de datos: Lógicos (booleanos)

AND Truth Table

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

XOR Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NOT Truth Tabl

A	B
0	1
1	0

Tipos de datos: Texto

Tipo	Ejemplo
`str`	`s = "Hola"`

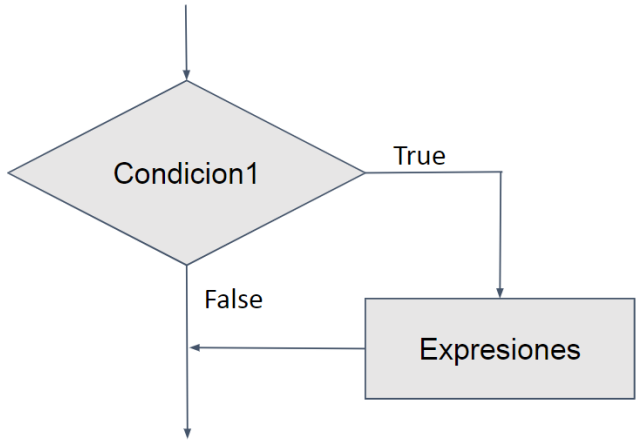
- Operaciones comunes

```
s = "Python"
s.lower()       # "python"
s.upper()       # "PYTHON"
s[0]            # "P"
s[-1]           # "n"
s[0:3]          # "Pyt"
len(s)          # 6
"Py" in s       # True
```

Instrucciones condicionales

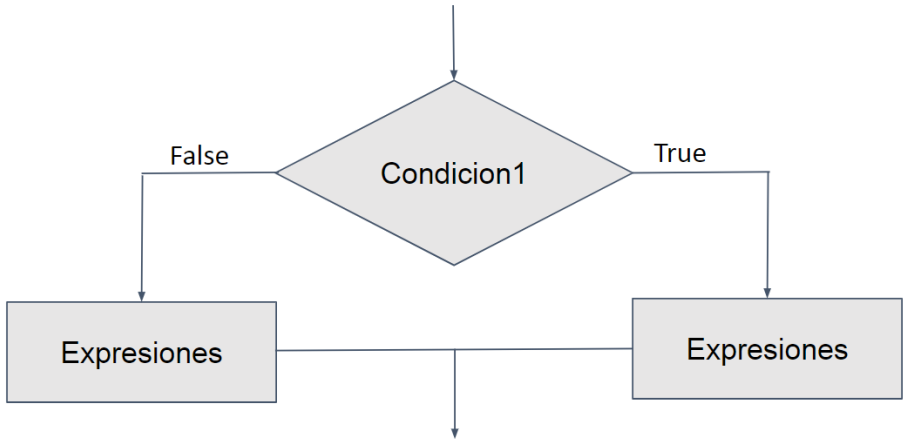

```
edad = 20

if edad < 18:
    print("Eres menor de edad.")
```




```
edad = 20

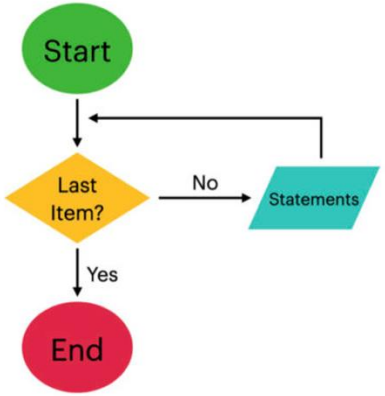
if edad < 18:
    print("Eres menor de edad.")
else:
    print("Eres mayor de edad.")
```



Instrucciones cíclicas

- Con **for**

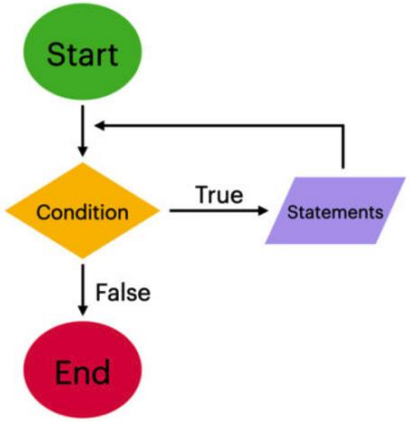
```python
# Ciclo for: recorre una secuencia de números
for i in range(5):  # range(5) genera 0,1,2,3,4
    print(f"Iteración número {i}")
```



- Con **while**

```python
# Ciclo while: se repite mientras la condición sea verdadera
contador = 1

while contador <= 5:
    print(f"Contador vale {contador}")
    contador += 1  # Incrementa en 1
```



Ejemplo a modelar

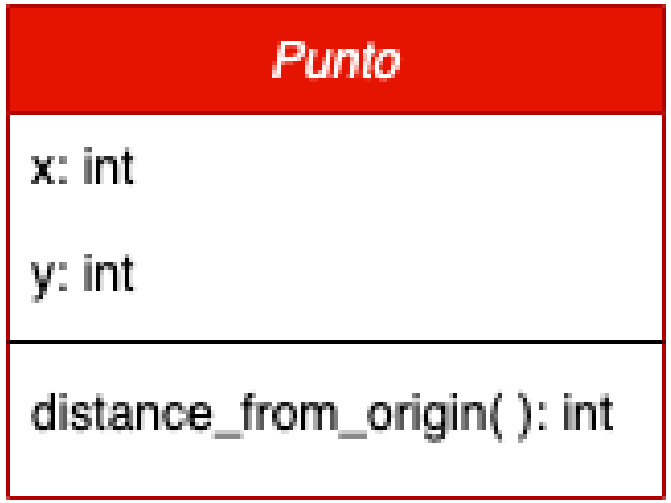
Punto matemático

Ejemplo a modelar

- En dos dimensiones, un punto son dos números (coordenadas) que se tratan colectivamente como un solo objeto.
- Los puntos a menudo se escriben entre paréntesis con una coma que separa las coordenadas.
- Ejemplo: **(0, 0)** representa el origen y **(x, y)** representa el punto **x** unidades a la derecha e **y** unidades hacia arriba desde el origen.
- Operaciones típicas asociadas con puntos:
  - calcular la distancia de un punto desde el origen
  - calcular la distancia de un punto desde otro punto
  - encontrar un punto medio de dos puntos
  - preguntar si un punto cae dentro de un rectángulo o círculo dado
  - …

Diagrama de clases

Ejemplo a modelar



Representación UML de la clase Punto

Definición de una nueva clase

Síntaxis de creación

Definición de una nueva clase

- Se inicia con `class <Nombre de la clase>`

```
class Point:
    """ Point class represents and manipulates x,y coords. """

    def __init__(self):
        """ Create a new point at the origin """
        self.x = 0
        self.y = 0
```



- `__init__` es el método inicializador, se llama cuando se usa la clase para construir un objeto
- El parámetro `self` es una referencia al objeto nuevo cuando se crea

Instanciación de objetos

Definición de una nueva clase

- Instanciar es usar la clase como plano para crear objetos.

```
p = Point()
q = Point()

print(p.x, p.y, q.x, q.y)
```

Atributos

Incluir datos en un objeto

- Se desea crear el siguiente objeto:

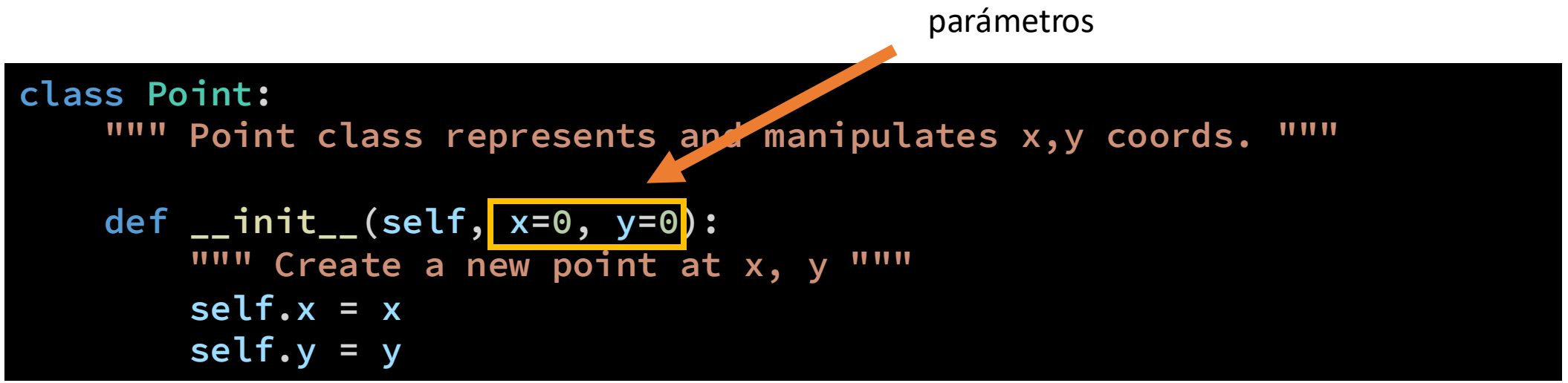


- Cada vez que se crea un punto hay que ejecutar estas tres líneas para cargarle datos

```
p = Point()
p.x = 3
p.y = 4
```

Mejorando el inicializador

- El **init** ahora incluye los parámetros



- Inicializando tres puntos

```
p = Point(4, 2)
q = Point(6, 3)
r = Point()
```

Operaciones o métodos

Definición de nuevos comportamientos

Operaciones o métodos

- Los métodos se crean como funciones
- Cuando llaman a algún atributo de la clase, se debe incluir el **self** como parámetro

```
class Point:
    """ Create a new Point, at coordinates x, y """

    def __init__(self, x=0, y=0):
        """ Create a new point at x, y """
        self.x = x
        self.y = y

    def distance_from_origin(self):
        """ Compute my distance from the origin """
        return ((self.x ** 2) + (self.y ** 2)) ** 0.5
```

Uso de instancias como parámetros

Una función que usa puntos

- El parámetro pt en la función es un objeto de tipo Punto

```
def print_point(pt):
    print(f"({pt.x}, {pt.y})")
```

Conversión de un objeto a string

Representación personalizada string

Conversión de un objeto a string

```
class Point:
    # ...

    def to_string(self):
        return f"({self.x}, {self.y})"
```

- La función permite conocer en detalle los datos importantes de la clase

```
print(p.to_string())
```

Uso de función str

Conversión de un objeto a string

```
class Point:
  # ...

  def __str__(self):
      return "({0}, {1})".format(self.x,
self.y)
```

- Renombrando el método se puede usar la función **str** directamente sobre el objeto

```
str(p)
print(p)
```

Instancias como valores de retorno

Ejemplo del punto medio

Se implementa primero como función

- Dados dos puntos encontrar el punto medio

```
def midpoint(p1, p2):
    """ Return the midpoint of points p1 and p2 """
    mx = (p1.x + p2.x)/2
    my = (p1.y + p2.y)/2
    return Point(mx, my)
```

- Uso de la función

```
p = Point(3, 4)
q = Point(5, 12)
r = midpoint(p, q)
r
```

Transformación a método

Instancias como valores de retorno

```
class Point:
    # ...

    def halfway(self, target):
        """ Return the halfway point between myself and the target """
        mx = (self.x + target.x)/2
        my = (self.y + target.y)/2
        return Point(mx, my)
```

- Se puede comprobar de varias formas

```
p = Point(3, 4)
q = Point(5, 12)
r = p.halfway(q)
r
```

```
print(Point(3, 4).halfway(Point(5, 12)))
```

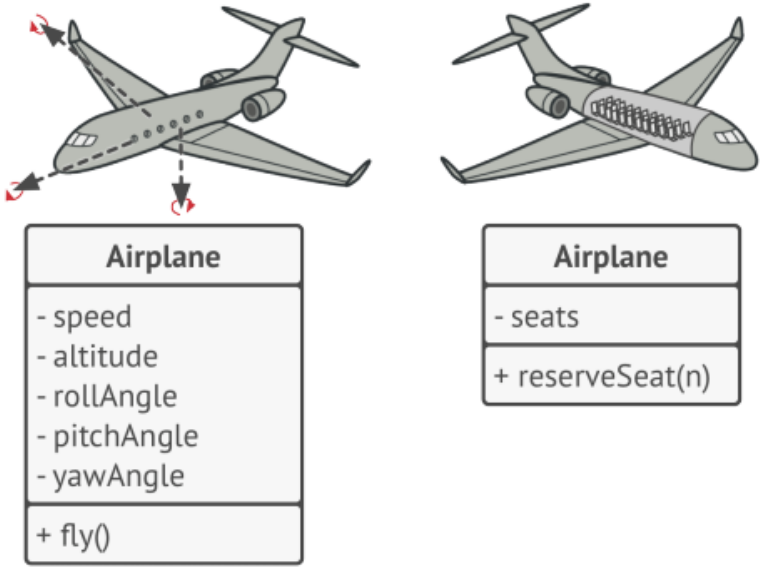
Características importantes de la programación orientada a objetos

Características de la programación de objetos

Detalles del modelado

- **Encapsulamiento**: evitar que cualquier código pueda modificar una estructura de datos

- **Abstracción**: propiedades relevantes a observar



Different models of the same real-world object.

Encapsulamiento


```
class Ejemplo:
    __atributo_privado = "Soy un atributo inalcanzable desde fuera."

    def __metodo_privado(self):
        print("Soy un método inalcanzable desde fuera.")

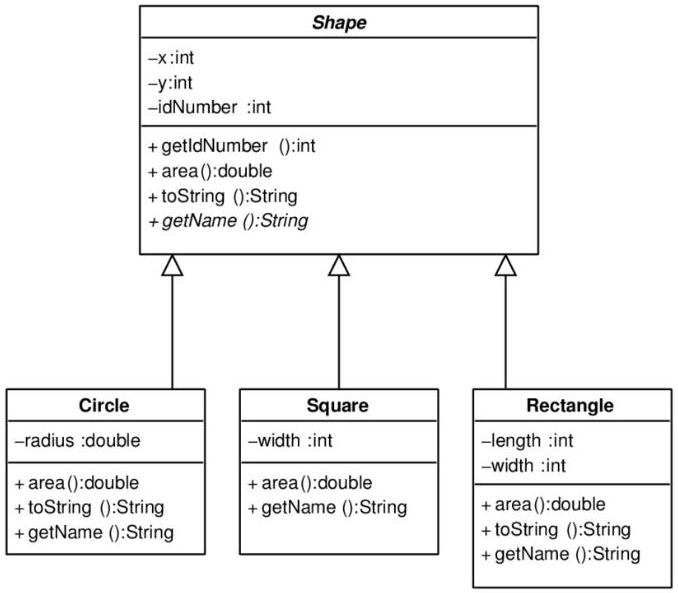
    def atributo_publico(self):
        return self.__atributo_privado

    def metodo_publico(self):
        return self.__metodo_privado()

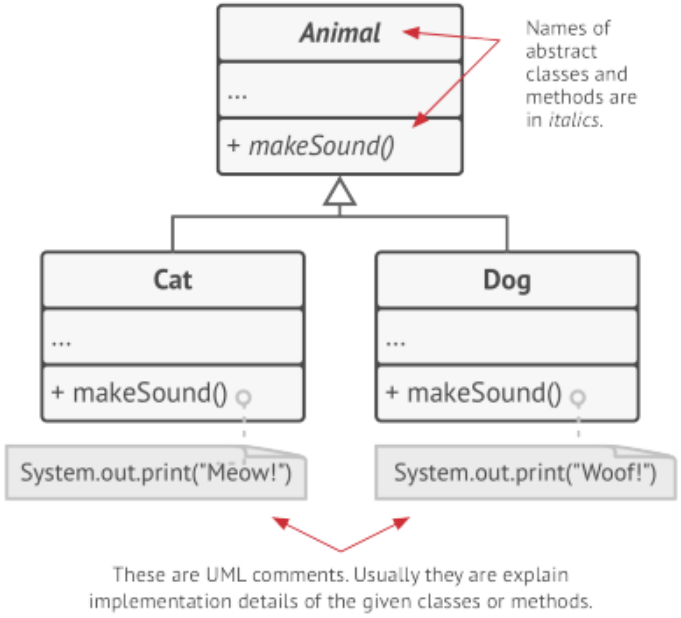
e = Ejemplo()
print(e.atributo_publico())
e.metodo_publico()
```

Características de la programación de objetos

- **Herencia**: reusar el código heredando el comportamiento de otros objetos



- **Polimorfismo**: el método invocado varía en función de la clase de la instancia de un objeto



Herencia


```
class Shape:
  def __init__(self,x,y):
    self.name = "Shape"
    self.x = x
    self.y = y

  def draw(self):
    print("Drawing",self.name,"at origin x:",self.x,"y:",self.y)

class Rectangle(Shape):
  def __init__(self,x,y,height,width):
    Shape.__init__(self,x,y)
    self.name = "Rectangle"
    self.height = height
    self.width = width

  def draw(self):
    print("Drawing",self.name,"at origin x:",self.x,"y:",self.y)
    print("Height:",self.height,"Width:",self.width)
```

Polimorfismo



```
class Animal():
    def __init__(self,name):
        self.name = name
    def speak(self):
        raise NotImplementedError("Subclass must implement this abtract method")

class Dog(Animal):
    def speak(self):
        return self.name + " says Woof!"

class Cat(Animal):
    def speak(self):
        return self.name + " says Meow!"
```

```
jim = Dog("Jim")
tim = Cat("Tim")
print(jim.speak())
print(tim.speak())
```

Gracias por tu asistencia y participación ☺

Contacto

miguela.orjuela@urosario.edu.co

https://www.linkedin.com/in/miguel-orjuela/

https://github.com/maorjuela73